SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)
Assessment Tool Weightage: 35%

Dr

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	**5**
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	**5**
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	**5**
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	**5**
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	**5**
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	**5**
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	**5**

8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	**5**
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	**5**
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	**5**

Code of Conduct:

I K.MOHAMMED NAFEEZ [192571070] certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *K.MOHAMMED NAFEEZ*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

ANSWERS – CO5 PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Q1. Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

Answer:

```
BEGIN
   INPUT SQL_Query

   Tokens ← LEXICAL_ANALYSIS(SQL_Query)
   ParseTree ← PARSE(Tokens)

   IF ParseTree is invalid THEN
      DISPLAY "SQL Syntax Error"
      STOP
   END IF

   LogicalPlan ← CONVERT_TO_RELATIONAL_ALGEBRA(ParseTree)
   OptimizedPlan ← OPTIMIZE_QUERY(LogicalPlan)
   PhysicalPlan ← SELECT_EXECUTION_OPERATORS(OptimizedPlan)

   Result ← EXECUTE(PhysicalPlan)
   DISPLAY Result
END
```

Main stages:

1. Parsing checks syntax and creates a parse tree.
2. Translation converts the query into a logical relational algebra plan.
3. Optimization improves the plan using cost or heuristic rules.
4. Physical-plan generation selects algorithms such as index scan, hash join, or nested-loop join.
5. Execution runs the physical plan and produces the result.

Q2. Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

Answer:

```
ALGORITHM CostBasedJoinOptimization(Relations, JoinConditions)

   BestPlan ← NULL
   BestCost ← INFINITY
```

```
    CandidatePlans ← GENERATE_JOIN_ORDERINGS(Relations, JoinConditions)

    FOR EACH Plan IN CandidatePlans DO
        Cost ← 0
        CurrentRelation ← FIRST_RELATION(Plan)

        FOR EACH NextRelation IN REMAINING_RELATIONS(Plan) DO
            OuterPages ← ESTIMATE_PAGES(CurrentRelation)
            InnerPages ← ESTIMATE_PAGES(NextRelation)
            Selectivity ← ESTIMATE_JOIN_SELECTIVITY(CurrentRelation, NextRelation)

            JoinCost ← ESTIMATE_JOIN_IO_COST(
                        OuterPages,
                        InnerPages,
                        Selectivity
                    )

            Cost ← Cost + JoinCost
            CurrentRelation ← ESTIMATE_JOIN_RESULT(
                            CurrentRelation,
                            NextRelation,
                            Selectivity
                        )
        END FOR

        IF Cost < BestCost THEN
            BestCost ← Cost
            BestPlan ← Plan
        END IF
    END FOR

    RETURN BestPlan, BestCost
END
```

The optimizer compares alternative join orders and selects the plan with the lowest estimated I/O cost.

Q3. Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

Answer:

```
ALGORITHM NestedLoopJoin(R, S)

  FOR EACH tuple r IN R DO
    FOR EACH tuple s IN S DO
      IF JOIN_CONDITION(r, s) = TRUE THEN
        OUTPUT COMBINE(r, s)
      END IF
    END FOR
  END FOR
END
```

For a block/page nested-loop join:

```
ALGORITHM BlockNestedLoopJoin(R, S)

  FOR EACH block BR of R DO
    LOAD BR INTO MEMORY
    FOR EACH block BS of S DO
      LOAD BS INTO MEMORY
      FOR EACH tuple r IN BR DO
        FOR EACH tuple s IN BS DO
          IF JOIN_CONDITION(r, s) THEN
            OUTPUT COMBINE(r, s)
          END IF
        END FOR
      END FOR
    END FOR
  END FOR
END
```

Complexity:

For tuple nested-loop join, approximately $O(|R| \times |S|)$ comparisons.

For block nested-loop join, with B(R) pages, B(S) pages, and M available buffer pages, the approximate I/O cost is:

B(R) + ceil(B(R)/(M-2)) × B(S)

Thus, increasing the number of buffer pages reduces repeated scans of the inner relation.

Q4. Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

Answer:

```
ALGORITHM DetectDeadlock(WFG)

    Visited ← EMPTY_SET
    RecursionStack ← EMPTY_SET

    FOR EACH transaction T IN WFG DO
        IF T NOT IN Visited THEN
            IF DFS_Cycle_Detection(T, Visited, RecursionStack) = TRUE THEN
                RETURN "Deadlock Detected"
            END IF
        END IF
    END FOR

    RETURN "No Deadlock"
END

FUNCTION DFS_Cycle_Detection(T, Visited, RecursionStack)

    ADD T TO Visited
    ADD T TO RecursionStack

    FOR EACH transaction U ADJACENT TO T DO
        IF U NOT IN Visited THEN
            IF DFS_Cycle_Detection(U, Visited, RecursionStack) = TRUE THEN
                RETURN TRUE
            END IF
        ELSE IF U IN RecursionStack THEN
            RETURN TRUE
        END IF
    END FOR

    REMOVE T FROM RecursionStack
```

```
    RETURN FALSE
END
```

A cycle in the Wait-For Graph means that transactions are waiting for one another, so a deadlock exists.

Q5. Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

Answer:

```
ALGORITHM Basic2PL(Transaction T)

    Phase ← GROWING

    FOR EACH operation O OF T DO

        IF O requires a lock THEN
            IF Phase = GROWING THEN
                IF LOCK_AVAILABLE(O.item) THEN
                    GRANT_LOCK(T, O.item)
                ELSE
                    WAIT(T, O.item)
                    WHEN LOCK_AVAILABLE(O.item):
                        GRANT_LOCK(T, O.item)
                END IF
            ELSE
                RETURN "Lock request rejected: transaction is in shrinking phase"
            END IF
        END IF

        EXECUTE(O)

        IF T decides to release a lock THEN
            Phase ← SHRINKING
            RELEASE_LOCK(T, O.item)
        END IF
    END FOR

    RELEASE_ALL_LOCKS(T)
END
```

Rules:

1. Growing phase: locks may be acquired, but not released.
2. Shrinking phase: locks may be released, but no new locks may be acquired.
3. The protocol ensures conflict serializability.
4. Basic 2PL may still cause deadlocks, so deadlock handling may be required.

Q6. Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

Answer:

For each data item X maintain:

```
   read_TS(X)  = largest timestamp of a transaction that successfully read X
   write_TS(X) = largest timestamp of a transaction that successfully wrote X

ALGORITHM ThomasWriteRule(T, X, TS(T))

   IF TS(T) < read_TS(X) THEN
      IGNORE WRITE(T, X)
      RETURN "Obsolete write ignored"
   END IF

   IF TS(T) < write_TS(X) THEN
      IGNORE WRITE(T, X)
      RETURN "Obsolete write ignored by Thomas' Write Rule"
   END IF

   PERFORM WRITE(T, X)
   write_TS(X) ← TS(T)
   RETURN "Write accepted"
END
```

For a read:

```
ALGORITHM TimestampRead(T, X)

   IF TS(T) < write_TS(X) THEN
      ABORT T
      RETURN "Read rejected"
   ELSE
```

```
    PERFORM READ(T, X)
    read_TS(X) ← MAX(read_TS(X), TS(T))
    RETURN "Read accepted"
  END IF
END
```

Thomas' Write Rule allows certain obsolete writes to be ignored instead of aborting the transaction, improving concurrency while preserving serializability.

Q7. Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.

Answer:

```
ALGORITHM ARIES_Recovery(Log, Checkpoint)

  PHASE 1: ANALYSIS
    Read the log from the last checkpoint.
    Reconstruct TransactionTable.
    Reconstruct DirtyPageTable.
    Identify transactions that committed.
    Identify transactions that were active at crash time.

  PHASE 2: REDO
    Determine the smallest LSN that may need recovery.
    Scan the log forward from that LSN.
    FOR EACH update record DO
      IF page requires REDO AND record is not already reflected THEN
        REAPPLY update
      END IF
    END FOR

  PHASE 3: UNDO
    LoserTransactions ← transactions active at crash time
    FOR EACH loser transaction DO
      Traverse its log records backward.
      Undo each applicable update.
      Write compensation log records.
    END FOR
```

```
    Write required completion records.
    RETURN "Recovery Complete"
END
```

ARIES therefore performs Analysis first, repeats necessary history during Redo, and reverses incomplete transactions during Undo.

Q8. Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

Answer:

```
ALGORITHM ShadowPaging

    shadowPageTable ← COPY(currentPageTable)
    transactionActive ← TRUE

    WHILE transactionActive DO
        READ requested page
        MODIFY page

        IF page is modified THEN
            newPage ← ALLOCATE_NEW_PAGE()
            WRITE_MODIFIED_DATA(newPage)
            currentPageTable[page] ← newPage
        END IF

        IF transaction COMMIT requested THEN
            WRITE_PAGE(currentPageTable)
            MAKE currentPageTable the committed page table
            transactionActive ← FALSE
            RETURN "Commit Successful"
        END IF

        IF SYSTEM_FAILURE THEN
            currentPageTable ← shadowPageTable
            transactionActive ← FALSE
            RETURN "Rollback Successful"
        END IF
    END WHILE
END
```

The shadow page table preserves the last consistent database state. If a failure occurs before commit, the system discards the modified current page table and restores the shadow page table.

Q9. Write pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

Answer:

```
ALGORITHM ImmediateUpdateRecovery(Log)

   Committed ← EMPTY_SET
   Active ← EMPTY_SET

   FOR EACH record IN Log DO
      IF record = START(T) THEN
         ADD T TO Active

      ELSE IF record = COMMIT(T) THEN
         ADD T TO Committed
         REMOVE T FROM Active
      END IF
   END FOR

   // REDO committed transactions
   FOR EACH record IN Log FORWARD DO
      IF record is an UPDATE by a transaction in Committed THEN
         REDO(record)
      END IF
   END FOR

   // UNDO uncommitted transactions
   FOR EACH record IN Log BACKWARD DO
      IF record is an UPDATE by a transaction in Active THEN
         UNDO(record)
      END IF
   END FOR

   RETURN "Recovery Complete"
END
```

Immediate Update may write changes to the database before commit, so recovery requires:

- REDO for committed transactions.
- UNDO for transactions that were active/uncommitted at the crash.

Q10. Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

Answer:

```
ALGORITHM DatabaseConnectivityManager

    INITIALIZE ConnectionPool
    SET MaximumConnections
    SET MinimumConnections

FUNCTION GET_CONNECTION()
    IF available connection exists THEN
        RETURN connection
    ELSE IF pool size < MaximumConnections THEN
        connection ← CREATE_CONNECTION()
        ADD connection TO pool
        RETURN connection
    ELSE
        WAIT until a connection becomes available
        RETURN available connection
    END IF
END

FUNCTION EXECUTE_QUERY(SQL, Parameters)

    connection ← GET_CONNECTION()

    TRY
        BEGIN TRANSACTION

        statement ← PREPARE(connection, SQL)
        BIND(statement, Parameters)

        result ← EXECUTE(statement)
```

```
        COMMIT TRANSACTION
        RETURN result

    CATCH Exception e
        ROLLBACK TRANSACTION
        LOG(e)
        RETURN "Query Failed"

    FINALLY
        CLOSE statement
        RETURN connection TO ConnectionPool
    END TRY
END

FUNCTION SHUTDOWN_POOL()
    FOR EACH connection IN ConnectionPool DO
        CLOSE connection
    END FOR
END
```

This manager reuses connections, executes parameterized queries, commits successful transactions, rolls back failed transactions, handles exceptions, and returns connections to the pool.